AI407L – AI Capstone Project Lab

Mid Exam Technical Report

Spring 2026

Name: _Anum Imran
Registration Number: 2022101

Course: AI407L – AI Capstone Project Lab
Instructor: Mr. Muhammad Naeem, M. Asadullah Amin

Institution: Ghulam Ishaq Khan Institute of Engineering Sciences & Technology

1. Introduction

Artificial Intelligence systems increasingly require interaction with external knowledge sources, reasoning frameworks, and operational tools. To design scalable and production-ready intelligent systems, modern architectures combine Retrieval-Augmented Generation (RAG), agent reasoning workflows, multi-agent collaboration, and modular tool integration.

This project implements an AI agent system using **LangGraph** along with a standalone **Model Context Protocol (MCP)** pipeline. The implementation follows the specifications defined in Lab 2, Lab 3, Lab 4, and Lab 5.

Part A focuses on building a complete **AI agent architecture**, including RAG-based knowledge retrieval, reasoning loops, multi-agent collaboration, and persistent memory with human-in-theloop safety mechanisms.

Part B focuses on implementing a **standalone MCP system** that exposes structured tools through an MCP server and demonstrates client-side tool discovery and invocation using the MCP protocol.

---

2. Part A – Agent System Implementation

Project repo: https://github.com/anum-imm/capstonelab-project

2.1 Retrieval-Augmented Generation (RAG) Pipeline

The system implements a **Retrieval-Augmented Generation pipeline** to ensure that responses generated by the model are grounded in domain knowledge rather than relying solely on model memory.

The RAG pipeline consists of the following components:

1. Knowledge Source
2. Document Embeddings
3. Vector Database
4. Retriever
5. Response Generation Model

RAG Workflow

User Query
↓
Embedding Model
↓
Vector Database Search
↓
Relevant Documents Retrieved
↓
LLM Response Generation
↓
Final Answer

This approach ensures that responses are context-aware and grounded in relevant knowledge sources.

2.2 LangGraph Reasoning Workflow

The reasoning workflow is implemented using **LangGraph**, which enables structured execution of reasoning steps using a graph-based architecture.

The workflow includes:

- Graph state definition
- Agent reasoning node
- Tool execution node
- Conditional routing logic

The agent follows a **ReAct (Reason + Act)** loop where the agent first reasons about the user query and then decides whether a tool needs to be executed.

Reasoning Workflow

User Query
↓
Agent Node (Reasoning)
↓
Tool Execution Node
↓

Agent Node (Reasoning)
↓
Final Response

The conditional routing mechanism determines whether the system should continue reasoning or return a final response.

2.3 Multi-Agent Architecture

The system is extended into a **multi-agent architecture** where different agents perform specialized tasks. Each agent has specific responsibilities and restricted access to tools.

Implemented Agents

1. **Research Agent**
   Responsible for retrieving and analyzing relevant knowledge from the RAG system.
2. **Analysis Agent**
   Responsible for reasoning about the retrieved information and generating the final response.

Multi-Agent Workflow

User Query
↓
Research Agent
↓
Knowledge Retrieval
↓
Analysis Agent
↓
Final Response

This architecture demonstrates collaborative problem solving where agents exchange state information and coordinate actions.

2.4 Persistent Memory and Human-in-the-Loop (HITL)

The system implements persistent memory using checkpoint-based state management. This allows the system to maintain session continuity and recover previous states using thread identifiers.

Additionally, a **Human-in-the-Loop (HITL)** safety mechanism is implemented to prevent execution of high-risk actions without human approval.

HITL Workflow

Agent proposes tool action
↓
Safety Check
↓
Human Approval Step
↓
Execute Tool / Cancel Action

This mechanism ensures that critical operations cannot be executed automatically without human verification.

---

3. Part B – Model Context Protocol (MCP) Pipeline

Part B implements a standalone **Model Context Protocol (MCP)** pipeline. The MCP system is implemented independently of the Part A agent system.

The MCP architecture separates model reasoning from tool execution using a server-client architecture.

MCP Architecture

User
↓
Model
↓
MCP Client
↓
MCP Server
↓
Tools
↓
Structured Response

3.1 MCP Server Implementation

A standalone MCP server was implemented to expose structured tools that can be accessed by a client model.

The server exposes two callable tools: 1.

Calculator Tool

2. **Temperature Converter Tool**

The tools are exposed using the MCP protocol through the @mcp.tool() decorator.

Example tool registration:

```
@mcp.tool()
def calculate(a: float, b: float, operation: str)
```

The MCP server is responsible for:

- registering available tools
- validating structured inputs
- executing tool logic
- returning structured responses

3.2 Structured Input and Output Schemas

Structured schemas were defined using **Pydantic models** to ensure proper validation of tool inputs and outputs.

Example schema:

```
class CalculatorInput(BaseModel):
   a: float     b:
float
operation: str
```

Structured schemas improve reliability and enforce data consistency during tool execution.

3.3 MCP Client Interaction Pipeline

The MCP client establishes communication with the MCP server and manages tool interactions.

The client performs the following operations:

1. Server connection
2. Tool discovery
3. Tool invocation

4. Response handling

Example output demonstrating tool discovery:

Available tools:
['calculate', 'convert_temperature']

Example tool execution:

User: add 2 and 4

AI Response:
{
 "result": 6
}

The client communicates with the server using the MCP protocol rather than directly calling functions.

Task 3 – Technical Comparison and Justification of MCP

Introduction

Modern AI systems increasingly require interaction with external tools such as APIs, databases, analytics engines, and automation services. As these systems grow in complexity, managing tool access in a secure, scalable, and maintainable manner becomes critical. The **Model Context Protocol (MCP)** provides a standardized architecture that allows AI models to access external tools through a structured client-server communication layer. This design separates model reasoning from tool execution and improves system modularity compared to traditional approaches.

Why MCP is Needed in Production Systems

In production environments, AI applications often interact with multiple services simultaneously. Directly embedding tool logic within model pipelines leads to tightly coupled systems that are difficult to maintain and scale.

The **Model Context Protocol (MCP)** solves this problem by introducing a standardized communication mechanism between models and tools. Instead of allowing the model to directly execute functions, MCP exposes tools through a dedicated server. The model communicates with this server through an MCP client, which manages tool discovery, invocation, and response handling.

This architecture provides better control over tool execution and allows tools to be updated or replaced without modifying the model logic. As a result, MCP enables scalable and maintainable AI systems suitable for production deployment.

Comparison of Tool Integration Approaches

1. Direct Tool Invocation

Direct tool invocation occurs when the application or model directly calls a function or API within the same program.

Example architecture:

User → Model → Function / API

Advantages

- Simple to implement
- Suitable for small-scale prototypes

Limitations

- Tight coupling between model and tools
- Difficult to scale when multiple tools are added
- Poor modularity
- Limited security controls

Because the model has direct access to tool implementations, system components become highly dependent on each other, making maintenance more difficult.

2. LangGraph-Based Orchestration

LangGraph is a framework used to orchestrate complex AI workflows. It organizes tasks into graph-based pipelines where nodes represent reasoning steps, tools, or agents.

Example architecture:

User → LangGraph Workflow → Tool Nodes → Response

Advantages

- Enables multi-agent systems
- Supports structured reasoning workflows
- Provides control over execution order

Limitations

- Tools remain embedded within the application environment
- Tool exposure is not standardized
- Workflow complexity increases as systems scale

LangGraph primarily focuses on managing reasoning workflows rather than providing a universal interface for external tool access.

3. MCP-Based Modular Tool Exposure

The Model Context Protocol introduces a modular architecture where tools are exposed through a dedicated MCP server.

Example architecture:

User
↓
Model
↓
MCP Client
↓
MCP Server
↓
Tools

Advantages

- Standardized tool exposure
- Clear separation between model and tool execution

- Tools can run independently of the model
- Supports modular and scalable system design

The MCP server acts as an intermediary that manages tool discovery, invocation, and structured responses.

How MCP Improves AI System Design

1. Security

MCP improves security by isolating tool execution from the model environment. Since tools are accessed through the MCP server, developers can enforce validation, authentication, and access control policies. This prevents unauthorized or unsafe tool execution.

2. Scalability

MCP enables scalable architectures by allowing tools to run as independent services. New tools can be added to the MCP server without modifying the client or model logic. This modular approach makes it easier to expand the system as requirements grow.

3. System Abstraction

MCP introduces an abstraction layer between the model and tool implementations. The model interacts only with tool descriptions and structured schemas rather than the underlying code. This abstraction simplifies system design and allows tools to be replaced or updated without affecting the model pipeline.

4. Separation of Concerns

MCP enforces clear separation between system components:

Component	Responsibility
Model	Interprets user queries and selects tools
Context	Maintains user input and interaction state
MCP Client	Handles communication with the MCP server

MCP Server	Registers and manages available tools
Tools	Perform the actual operations or computations

This separation improves system maintainability and allows developers to update individual components independently.

Conclusion

The Model Context Protocol provides a structured and scalable method for integrating external tools into AI systems. Compared to direct tool invocation and orchestration frameworks such as LangGraph, MCP offers stronger modularity, improved security, and better separation of system responsibilities. By exposing tools through a standardized server interface, MCP enables production-ready AI architectures that are easier to maintain, scale, and secure.